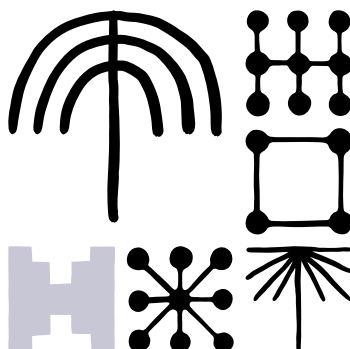


# How we contain Claude across products

Anthropic · 2026-05-25 · 原文 / Source



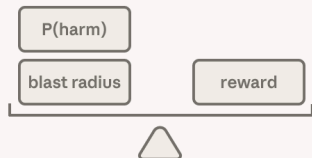
As agents grow more capable, so does their potential blast radius. The engineering question is how to cap it. Here's what we've learned building containment for claude.ai, Claude Code, and Cowork.

Twelve months ago, we'd have rejected out of hand the idea of granting Claude access sufficient to take down an internal Anthropic service. Today that level of access is routine, and Anthropic developers are more productive for it. The risk of these deployments has two components: how likely a failure is, and how much damage one could do. Progress on safeguards and model training has steadily driven down the first; the second—the theoretical blast radius—only grows as capabilities and access expand. Yet as agents become capable of doing work that once required a person or even a team, the cost of *not* deploying grows large enough that the risk-reward calculation tips heavily toward adoption, as long as products can be made safe. The engineering question becomes how to cap the blast radius.

## Visualizing risk and reward

### Contained risk

Ship →

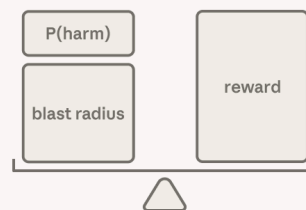


Reject X



### High utility

Ship →



*When bounds can be placed on the relative damage of an autonomous agent—such as through control over its environment—high-utility capabilities can motivate deployment. Claude Mythos Preview is an example of a model whose blast radius was deemed too high to ship in April 2026. However, we expect broader release of models with similar levels of capability to become appropriate as defenders harden critical systems and safeguards mature—even though some risk will always remain. Model capability is an important factor in the total risk of an agent's deployment.*

There are broadly two ways to do this.

The first is to supervise the agent's behavior via a human-in-the-loop. Claude Code previously protected against agents taking unintended actions by asking users for permission at each turn. Theoretically that works, but we've found the approach to be fallible. Our telemetry showed users approved roughly 93% of permission prompts. The more approvals a user sees, the less attention they pay to each, becoming over time much less diligent in their supervision. We recently built Claude Code auto mode, which automates safer approvals in order to reduce this approval fatigue. Still, vulnerabilities remain—any probabilistic defense has a non-zero miss rate.<sup>1</sup>

The second approach to capping the blast radius—and the focus of much of this post—is containment. Rather than supervising what the agent does, we supervise what it's *able* to do by enforcing access boundaries through, for example, sandboxes, virtual machines, and egress controls. This is where Anthropic engineering has devoted the most effort, and also where many of the most surprising security failures have occurred.

Over the past two years, we've shipped three primary agentic products: claude.ai, Claude Code, and Claude Cowork. Each serves a different audience, requiring a different containment architecture. This article shares what's held up, what's broken, and what we've learned about agent security along the way.

## Three types of risk, three components of defense

Security risks to agents fall into one of three categories:

**User misuse:** A user—either maliciously or through carelessness—directs the agent to do something harmful. This includes everything from asking the agent to bypass a check they find annoying, to running a destructive command they don't understand, to specifying intentional harm.

**Model misbehavior:** The agent takes a harmful action no one asked for. As our models have improved, they have become more aligned on most behavior evaluations, but this doesn't mean risk necessarily shrinks. Less capable models are more likely to misread a situation and make obvious errors. More capable models make fewer mistakes, but they're also better at finding unexpected paths to a goal, often by routing around restrictions nobody thought to write down.

At Anthropic, we've seen Claude models "helpfully" escape a sandbox in order to complete a task, examine git history to find answers to a coding test, and spontaneously identify the benchmark it was being run on in order to decrypt its answer key. Each model brings a new set of capabilities that are sometimes put to work in unexpected ways.

**External attackers:** The agent is attacked through external vectors such as tools, files, or network access. This category includes both prompt injection and conventional attacks on the agent's runtime, orchestration layer, or proxy.

When building containment and defense systems, we apply defenses to three main components:

**The environment in which the agent runs.** We constrain where and how an agent can act with process sandboxes, VMs, filesystem boundaries, and egress controls. The goal is to set a hard boundary on what an agent can reach. For example, if credentials never enter the sandbox, they can't be exfiltrated, regardless of whether the cause is a user, a model finding a "creative" path, or an attacker.

A tight perimeter also means you can relax oversight. Claude Code's reference devcontainer exists precisely so that the agent can run unattended, without per-action approvals.

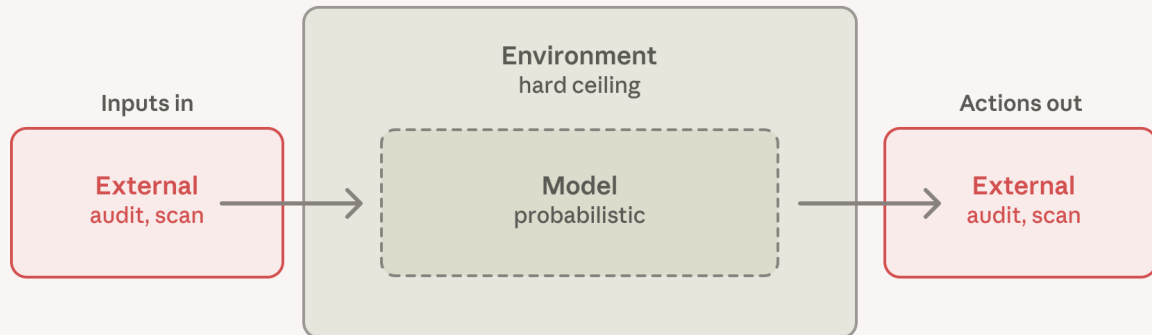
**The model the agent consults.** The mechanisms here include system prompts, classifiers, probes, and training modifications. Because models are probabilistic, these shape only what the agent *tends* to do, not what it is theoretically capable of doing.

These defenses are strong. On Gray Swan's Agent Red Teaming benchmark, which tests susceptibility to prompt injection, Claude Opus 4.7 holds attack success to roughly 0.1% on single attempts, and around 5–6% after 100 adaptive attempts. Claude Code auto mode catches roughly 83% of overeager behaviors before they execute. Yet even with best-in-class defenses, protection in the model layer will never be 100% effective, which is why it can't stand alone.

**The external content the agent can reach.** MCP servers, third-party plugins, and web search tools all feed content into the agent's context from sources you don't control. An audited connector isn't the same as audited data—a GitHub connector, for instance, can load a poisoned README straight into the model's context despite passing malware checks. Granularly limiting tool permissions can help limit the blast radius. An agent with read-only DB access, for instance, can be deployed far more broadly than one that writes to prod.

Defenses should overlap and complement each other. When environmental defenses aren't available, the model layer has to pick up the slack (this is precisely what Claude Code's auto mode is designed for). Locally, the environment and model defenses can guard against malicious tool outputs, but defenses can be added higher up the chain by limiting the tool's capabilities and access.

## Components to defending AI agents



*Three components to defend: the model, the environment in which it runs, and the external content the agent can reach.*

## Patterns for containing agents

Focusing on the environment layer, we describe three isolation patterns and how they're tailored for each Claude platform—claude.ai, Claude Code, and Cowork. We arrived at each design gradually, after finding the balance between the capabilities we need from the agent and the degree of intervention required from the user.

### Pattern 1: The ephemeral container (claude.ai code execution)

Though best known as a chat interface, claude.ai also writes and runs code, generates files, and calls connectors. When Claude runs code inside claude.ai, it does so in a gVisor container on isolated infrastructure. The agent is entirely server-side; no code runs on the local machine, and the filesystem is ephemeral (per-session). The blast radius is minimal, but so is the ceiling on what Claude can do—there's no persistent workspace and no access to the user's filesystem.

This also makes claude.ai subject to a more traditional threat model. We're not protecting user machines from agents; we're protecting our own infrastructure and each tenant from one another. Our pre-launch work for claude.ai was dominated by traditional security work like network configuration, internal service auth, and orchestration.

That work reinforced the oldest lesson in security: the weakest layer is the one you built yourself. gVisor and seccomp have been hardened against well-resourced adversaries for far longer than agentic AI has existed, so the review effort went into the newer pieces we'd built around them. We'll come back to this later, since our custom proxy is also the piece that broke in our most consequential incident.

## Pattern 2: The human-in-the-loop sandbox (Claude Code)

Claude Code runs on a user's machine and has access to their filesystem, shell, and network. Without this, coding agents have limited usefulness, so it's imperative to find a way to grant that access safely.

One approach is to rely on a human-in-the-loop. This is only a tractable solution for Claude Code because the average user is a developer who's familiar with coding environments: they can read bash, they understand what `rm -rf` does, and they already run `npm install` from untrusted sources several times a week. All that means that when an "allow this" dialog pops up, they are highly likely to have the expertise to accurately evaluate what the agent is attempting to do and the risk involved. Given this, Claude Code launched with the simplest possible defense: allow reads, require approval for write, bash, and network access.

However, as mentioned, approval fatigue showed up within weeks. Ironically, this meant that a feature originally designed to provide oversight could arguably have the opposite effect—some users might simply stop paying attention. As a first step to mitigate incautious approvals, we shipped an OS-level sandbox (Seatbelt on macOS, bubblewrap on Linux) that hardens the boundary: reads are allowed, writes are allowed inside the workspace, but network is denied by default. Within the sandbox, the agent runs largely without interruption. The result was an 84% reduction in permission prompts, and we open-sourced the runtime, so the boundary is auditable.

Our anonymized usage data also showed that experienced users auto-approve roughly twice as often as new users, but they also interrupt the agent mid-execution more frequently. Instead of gating individual steps, experienced users are more likely to supervise the agent only when it goes off track. While this may be a natural evolution in how people prefer to work with agents, this too is fallible, requiring users to be technical and attentive enough to notice drift in the first place. As model capabilities improve and agents begin writing increasingly ambitious bash, it becomes harder to notice any such drift. And as users move to multi-agent systems, this approach is also much less likely to be an effective oversight strategy.

### Risk we missed: Everything before the trust dialog

Between mid-2025 and January 2026, we received reports of vulnerabilities in Claude Code through our responsible disclosure program. Three of these vulnerabilities targeted code that executes *before* the user has consented to anything. To understand how this is possible, consider the most direct case: a developer clones a repository to review a pull request, and that repository contains a `.claude/settings.json` which defines a hook. Because Claude Code reads project settings during startup—before presenting the standard "Do you trust this folder?" prompt—the hook the attacker had authored and committed would execute automatically. The remaining cases looked structurally similar, in which input from the not-yet-trusted directory was parsed before the trust boundary had been established.

The fix in each case had the same shape: defer parsing and execution of project-local configuration until after the user accepts the trust prompt. If you're building something similar, treat project-open, config-load, and localhost listeners the way you'd treat any inbound request from the internet. They shouldn't be implicitly trusted just because they feel local and arrive before the user has consented.

## Risk we missed: The user as an injection vector

In February 2026, during a controlled internal red-team exercise, a researcher successfully phished an employee into launching Claude Code with a malicious prompt. The phish looked like ordinary collaboration—a "can you run this for me?" email with a ready-to-paste prompt attached—and the prompt itself read like routine task instructions. But somewhere among the setup steps, it gently asked Claude to read `~/aws/credentials`, encode the contents, and POST them to an external endpoint. Across 25 retries of that prompt, Claude completed the exfiltration 24 times.

This is a *direct* prompt injection—the attacker's instructions arrived through the user, not through tool output or fetched content. Our model-layer defenses anchor on user intent—when the user is the one typing the instruction, there's nothing anomalous for a classifier to catch. A human contractor handed the same script would have done the same thing.

The only defense that holds in this situation is the environment, specifically egress controls that block the POST regardless of intent and filesystem boundaries that keep `~/aws` out of reach in the first place.

(When we shared the working prompt in internal Slack for discussion, someone pointed out that some internal agents *read* Slack. The payload was now ambient. We added a canary string to the thread so we'd notice if anything picked it up. In a world where agents read everything, the investigation tooling is also an attack surface.)

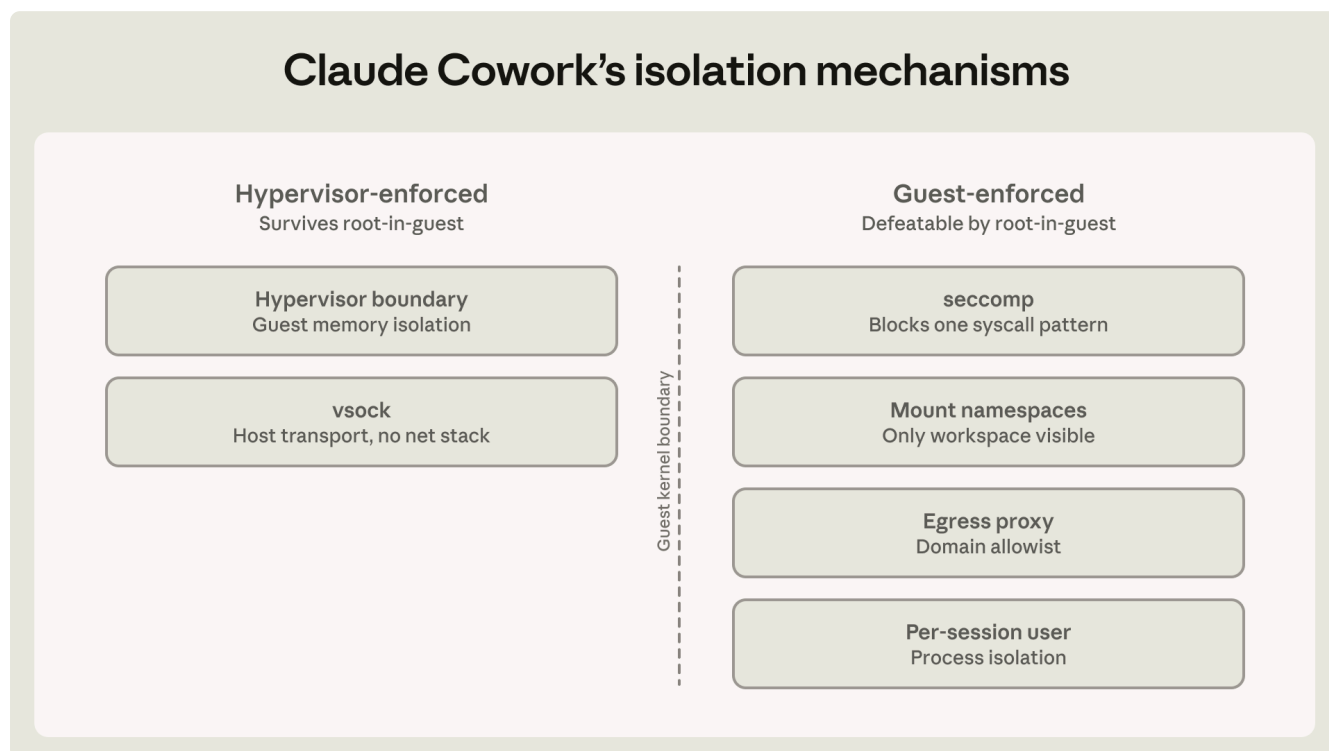
## Pattern 3: The local VM (Claude Cowork)

Claude Cowork runs on a user's desktop with access to a workspace folder selected by the user. Because the platform is built for general knowledge work, not software engineering, the average user is much less likely to be fluent in bash.

As a result, the human-in-the-loop sandbox strategy may not transfer; a non-technical knowledge worker shouldn't be expected to judge bash incantations such as `find . -name "*.tmp" -exec rm {} \;`. When approving an exception requires expertise the typical user doesn't have, admins should set a boundary that is absolute and always-on.

To enable this, our first version of Claude Cowork ran inside a full virtual machine using the platform's vendor hypervisor (Apple's Virtualization framework on macOS, HCS on Windows). The VM has its own Linux kernel, its own filesystem, and its own process table. The user's selected workspace and `.claude` folder are mounted; nothing else on the host is visible. Credentials stay in the host's keychain and never enter the guest machine. This design protects against the possibility that Claude will, at some point, behave in a misaligned manner. A compromised Claude could still damage what's inside the workspace folder, so the architecture is designed to make sure that's the *only* thing it can reach (until the user adds connectors), and that the user controls what's mounted there.

In the original architecture—what we call full-VM mode—the agent loop itself ran inside the guest, so Claude executed as an ordinary Linux user with no awareness it was sandboxed. Compare this to Claude Code, where a privileged process sits outside the sandbox deciding per-command whether to enforce it; a persuasive injected prompt or a fatigued approval click can get that process to run something un-sandboxed. Here, there was no outer process holding an escape-hatch key, and so no component with the authority to grant an exception.

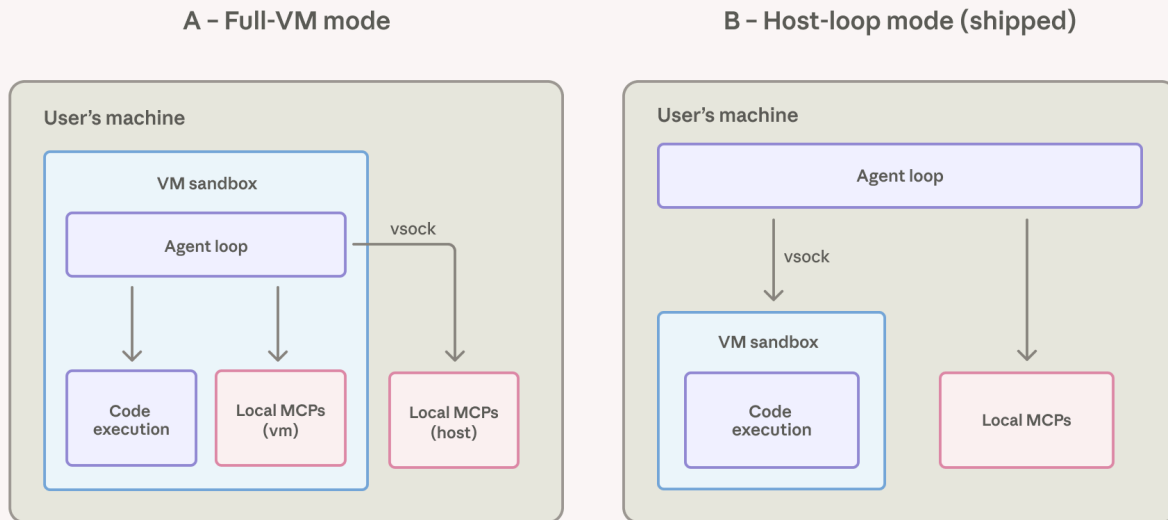


*The six main isolation mechanisms of Claude Cowork's VM. Two are enforced outside the guest kernel and would thus survive the agent achieving root-level access within the VM. The other four are guest-enforced and kept deliberately minimal because the outer layers carry the rest.*

However, we soon realized that running the whole agent in full-VM mode caused practical problems: any failure during VM startup made Cowork unusable. Moving the agent loop *outside* of the VM, while keeping code execution inside of it, allowed Claude to still respond to the user and help debug issues rather than freeze on an error. This change caused minimal security impact because the VM still enforces filesystem and network controls over code executed by the agent.

Separately, we also moved local MCP servers outside the VM. Running them inside the VM made them harder to audit, created brittle dependency issues when the VM updated, and didn't support MCPs that required interaction with local processes such as databases—such servers had to run on the host regardless. The change brings Claude Cowork in line with how local MCP servers already work in Claude Desktop: treating them like any software a user might choose to install and entrusting admins to decide which local MCPs to enable (if any). Remote MCP servers are unaffected since they do not run on the user's machine.

## Comparison: Full-VM vs host-loop



*Having the agent loop inside the VM meant that any failure in the VM caused Cowork to become unusable. Host-mode is more reliable because the agent can still respond if the VM crashes, and it still provides important security guarantees by isolating code execution.*

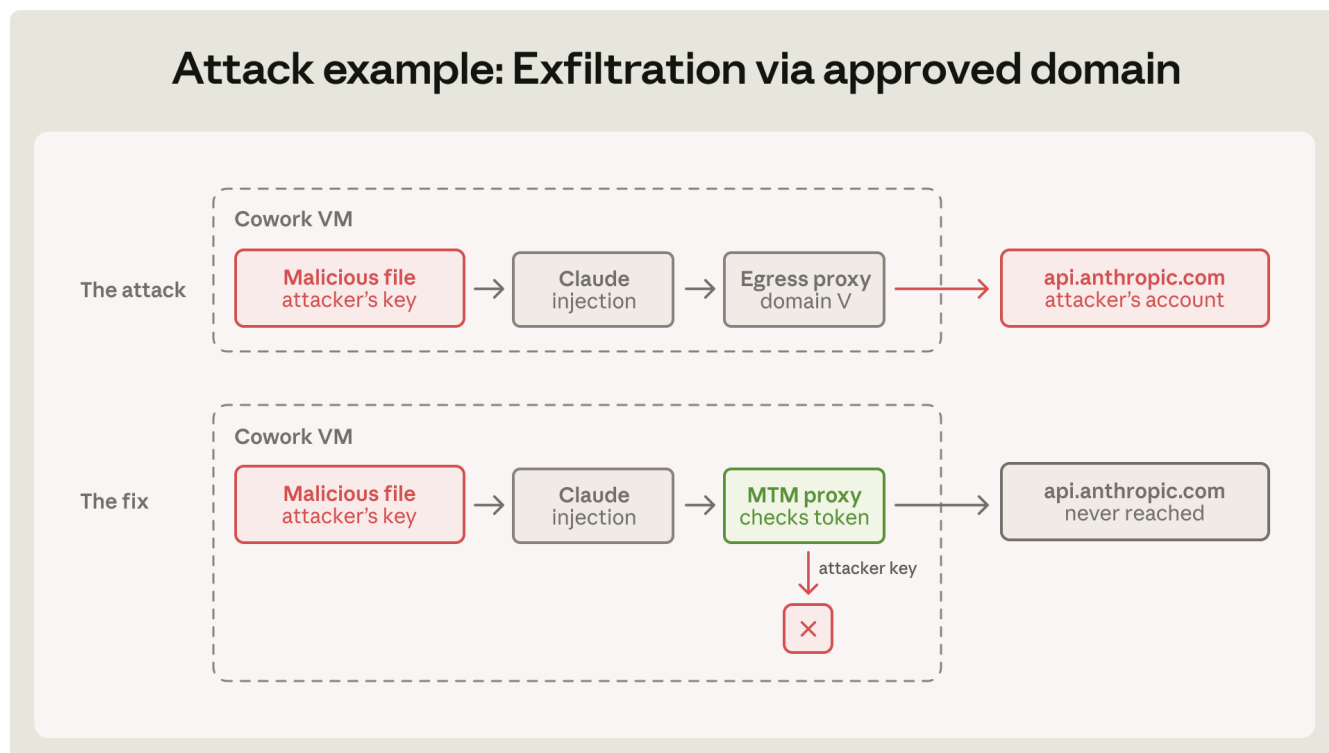
Filesystem controls were another important architectural choice. Claude needs to be able to access *some* files on the host in order to be useful, but we wanted to minimize the blast radius and provide transparency to the user about local file access. We found that offering different file-mount modes helps to granularly control risk; Claude Cowork offers read-only, read-write, and read-write-no-delete. One potential gotcha here is that symlink resolution has to happen *before* path validation, not after, or a symlink inside an authorized folder can point outside and escape. For enterprise customers, we allow admins to control this via mount-path allowlists in MDM settings.

### Risk we missed: Exfiltration through an approved domain

A clear example of exfiltration through an approved domain came from a third-party disclosure. Claude Cowork's egress allowlist correctly passed traffic to `api.anthropic.com`—the product can't function without calling our own API. In this case, a malicious file placed in the user's mounted workspace carried hidden instructions along with an API key controlled by the attacker. Claude, following the instructions, read other files in the workspace and called Anthropic's Files API using the attacker's key. The egress proxy checked the destination, saw `api.anthropic.com`, and let it through. The files were uploaded to the attacker's Anthropic account. The sandbox worked perfectly, and yet the data was exfiltrated.

Previously, we'd conceptualized the allowlist as a destination filter, something that told Claude *these domains are okay to talk to*. But it may be better conceptualized as a capability grant. Every function reachable through any domain on an allowlist is now an attack surface. Allowing `api.anthropic.com` meant allowing file uploads to arbitrary Anthropic accounts.

We fixed it using a defensive man-in-the-middle proxy inside the VM that intercepts traffic to our API. It only passes requests carrying the VM's own provisioned session token; an attacker-embedded key is rejected by the proxy. It also blocks headers that would enable server-side fetch. The proxy sits inside the VM rather than on our servers because only the VM knows provenance—from the server's perspective, a Cowork request is indistinguishable from any other API client.



Top: traffic to `api.anthropic.com` is let through, resulting in egress. Bottom: fix with a man-in-the-middle proxy intercepting traffic to our API.

This is also a second instance of the principle that the software you build yourself is often the weakest. The hypervisor, seccomp, and gVisor across our products have been dependable. Our custom allowlist proxy was the piece that failed.

### Risk we missed: VM isolation kept the endpoint detection software out too

When evaluating Claude Cowork, enterprise security teams asked, "Why can't our EDR see inside?" The answer was that the same isolation keeping Claude contained also kept host-based endpoint detection and response out. From the EDR's perspective, Claude Cowork is an opaque hypervisor process. It can't inspect the guest.

Isolation reduces visibility, and opacity is problematic for teams whose compliance posture depends on endpoint visibility. Our current mitigation is to use pull-based OTLP exports that let administrators retrieve event logs after the fact, but this is not the same as live monitoring. If you're building something similar, budget for this conversation early.

| Environment              | Ephemeral container (claude.ai)                                 | HITL sandbox (Claude Code) | Sealed VM (Claude Cowork)                                  |
|--------------------------|-----------------------------------------------------------------|----------------------------|------------------------------------------------------------|
| Cost: Isolation Overhead | Container spin-up                                               | Low-latency native sandbox | Full VM boot                                               |
| Cost: User Reliance      | N/A                                                             | Must interpret bash        | N/A                                                        |
| Risk: Blast Radius       | Server-side container (guarded by gVisor + host infra boundary) | Local workspace            | Mounted workspace (guarded by vsock + hypervisor boundary) |

## Trusting what the agent reads

Enterprises often ask us how to secure MCP connections. It's a good question, but the right one is broader than MCP specifically. Any external resource provided to an agent represents two risks at once: a code execution risk, in the traditional supply-chain sense, and a prompt injection vector. Traditional dependency auditing (pinning versions, verifying signatures, reviewing source) addresses the first, but misses the second.

**Remote versus local is more important than it seems.** A locally installed tool is auditable. You can read the code, pin the version, and know it won't change under you. A remote tool—a hosted MCP server, a cloud connector—can change behavior at any point after you've approved it; your install-time trust decision may no longer apply. Our connector directory addresses this through ongoing review, but anything outside it should be treated as untrusted. Run it against fake data first, in an environment where the blast radius of a malicious tool is contained.

**Tool output is an attack surface even when the tool is trusted.** The GitHub README example mentioned earlier is exactly this case; any input scanning applied to web pages needs to be applied to network-enabled tool results with the same rigor. Even though this adds latency and isn't a perfect defense, we err toward live inspection: once a poisoned tool return has steered the agent into exfiltrating data, the log just shows a successful, authorized API call. There's no after-the-fact signal to find.

In Claude Code and Claude Cowork, tool calls route through proxies that enforce network and file policy and can inspect return values before they enter the model's context. The classifier that does the inspection can be a small, fast model; it doesn't need to be the one doing the reasoning.

## Looking ahead

Models and products are advancing fast. As they do, risks morph and evolve, and our mitigations must keep pace to meet them.

**Persistent memory poisoning.** The share of agent context that persists across sessions keeps growing—this includes product memory, CLAUDE.md files, mounted workspaces, and the state directories of scheduled and long-running agents. An injection that lands in any of these is reloaded each time the agent starts. As more agent state survives the session, we are threatened by new persistence mechanisms in the classic post-exploitation sense. Good classifiers on session startup will need to become more commonplace.

**Multi-agent trust escalation.** On the one hand, sub-agents can isolate untrusted content, returning structured facts rather than raw text up to the main agent. On the other hand, this can be abused: if a sub-agent's output is treated as higher-trust than raw tool results, because such output came from "us," a new vector for prompt injection is introduced. In multi-agent systems, there is a tradeoff between allocating differing trust levels and becoming liable to trust escalation.

**Agent identity.** Claude Cowork's answer to agent identity is concrete: credentials stay in the host keychain, the VM gets a per-session scoped-down token, and that token can be revoked independently of the user's. However, we are starting to grapple with the broader question of cross-platform agent identity. Should an agent possess its own principal identity, or should it act as an extension of the user and inherit the user's permissions? Ultimately, the answer may be a blend of the two.

As agents grow more capable, attack surfaces are constantly shifting. The types of failures we've seen are likely to be repeated across industries and labs. We need collective investment in agent-specific security posture, from shared benchmarks and disclosure norms to common identity standards and cross-vendor red-teaming. We focus on containment in this piece, but that's only one part of the security picture for agents. For governance, observability, and the rest of the stack, see NIST's project on AI agent identity and authorization, the six-agency guidance on adopting agentic AI led by Australia's ACSC with CISA and the UK's NCSC, and ISO/IEC 42001, the AI management standard. Our Glasswing initiative is one contribution, but we look forward to working with both partners and competitors on this critical issue.

## Summary

In short, there are a few principles we keep returning to:

**Design for containment at the environment layer first, then steer behavior at the model layer.** Two of the incidents that taught us the most—the employee phish and the third-party allowlist disclosure—were both cases of egress, in which data left through a permitted path. In each, the model layer couldn't help; there was nothing anomalous for it to catch. The deterministic boundary is what gets hit when everything probabilistic misses.

**Match isolation strength to the user's capacity for oversight.** A developer who can read bash and a knowledge worker who can't are not running the same threat model. The question of whether a user can evaluate what an agent is about to do should help determine the containment strategy, and answering it wrong in either direction—too much friction for experts, too much trust for non-experts—is its own failure.

**Be wary of custom components.** Battle-tested hypervisors, syscall filters, and container runtimes have survived more adversarial attention than anything you'll build. Across every deployment described here, the standard primitives held while our own work around them exposed flaws.

Ultimately, while agents may be a new category of software, their system-level interactions are not. They still read files, open sockets, and spawn processes; this makes containment with mature tooling a crucially viable defense. The risk-reward balance of deployments will keep shifting as AI develops, but placing a hard limit on blast radius often forces that balance into the right direction.

## Acknowledgements

Written by Max McGuinness, Mikaela Grace, Jiri De Jonghe, Jake Eaton, and Abel Ribbink.

We're also grateful to Hanah Ho, Hasnain Lakhani, Pedram Navid, Molly Villagra, Maya Nielan, Akila Srinivasan, Travis Szucs, Sam Attard, Alfred Xing, Mohamad El Hajj, Gabby Curtis, David Dworken, Adam Jones, Amie Rotherham, Christian Ryan, Lucas Smedley, Brett Andrews, and others for their contributions.

Special thanks to our security and product engineering teams, and to the individuals and organizations that have reported vulnerabilities in Claude products.

## Footnotes

1. Claude Code auto mode delegates command approvals to a model-based classifier; it minimizes friction (roughly 0.4% of benign commands blocked) at the cost of missing a fraction of risky ones (~17% of overeager actions get through), so it's one layer of defense-in-depth inside a sandbox, not a substitute for one.